SMART DAILY REMINDER ASSISTANT

An Offline IoT System Using ESP32 and Edge Computing

Jaydeep Parmar — 23BIT133
Tadvi Yatharth — 23BIT143
Dev Mehta— 23BIT145

1. Introduction

The Smart Daily Reminder Assistant is an embedded Internet of Things (IoT) device designed to manage daily schedules, classes, examinations, assignments and personal events. It is built around an ESP32 development board and follows an edge-computing approach: reminder data is stored and processed locally by the device instead of being sent to a remote server. The system provides three ways to interact with reminders: a browser-based dashboard hosted by the ESP32, a 0.96-inch OLED display for viewing upcoming events, and physical buttons for acknowledging, postponing and navigating alerts. A passive buzzer provides audible notifications at the scheduled time.

2. Problem Statement

Many reminder applications depend on a smartphone application, an internet connection, or cloud infrastructure. Such dependencies can reduce availability when connectivity is weak or unavailable and may require personal schedule information to be stored on external services. A desk-based reminder device should continue to operate locally, provide clear audible and visual alerts, and allow the user to manage a timetable without relying on an internet service.

3. Project Objectives

- Develop a standalone reminder assistant using the ESP32 microcontroller.
- Store reminder records in the ESP32's internal flash using the LittleFS file system.
- Provide a local web interface for adding, viewing, importing and managing reminders.
- Display the next reminder and system information on an OLED screen.
- Generate repeated alarms and provide STOP, SNOOZE and MENU controls.
- Keep reminder storage, scheduling and alert processing on the device, without cloud services or internet access.

4. Uniqueness and Novelty

The main novelty is the integration of reminder management, local data storage, scheduling, a web dashboard and physical alert controls into one low-cost embedded device. The ESP32 is not merely a Wi-Fi client connected to an external application: it acts as the local access point, web server, data store and scheduling controller.

- Completely local operation: the device creates its own Wi-Fi hotspot, so a phone or laptop can connect directly without a router or internet connection.
- Device-resident data: reminders are saved as a readable JSON file in LittleFS flash and remain available after a restart.
- Purpose-built alert logic: an ignored reminder can trigger up to three 30-second alarms, five minutes apart; one STOP press ends the sequence permanently for that occurrence.
- Multiple interaction modes: the browser dashboard is complemented by dedicated physical controls and an OLED display.
- Efficient timetable representation: daily and weekly repeat settings avoid entering a separate record for every repeated class.

5. System Architecture and Working

The ESP32 is the central controller. It hosts a Wi-Fi access point named “SmartReminder” and serves the dashboard at the local address http://192.168.4.1. A phone or laptop joins this hotspot and uses its browser to communicate directly with the ESP32. The connection is local Wi-Fi only; the project does not use a cloud database, cloud platform, internet API, or external internet connection.

5.1 Main Functional Blocks

- Local Wi-Fi access point and web server: provides the direct connection and serves the dashboard stored in the firmware.
- Dashboard: allows the user to create individual reminders, bulk-import timetable entries, view reminder status, mark items done, delete entries and clear completed or missed records.
- LittleFS storage: saves reminder information in /reminders.json in the ESP32 flash memory.
- Scheduler: compares the current device time with reminder times, selects the earliest due reminder and calculates the current alert stage.
- OLED and buzzer: present event information and produce audible alerts.
- Three push buttons: STOP acknowledges an alert, SNOOZE postpones it, and MENU cycles through OLED pages.

5.2 Operating Sequence

1. Power on the ESP32. It initializes the display, buttons, buzzer, LittleFS storage, Wi-Fi access point and local web server.
2. Connect a phone or laptop to the SmartReminder Wi-Fi network and open http://192.168.4.1 in a browser.
3. When the dashboard loads, the browser sends the phone's current time to the ESP32 through the local connection. The ESP32 sets its system clock; no internet time service is used.
4. Enter reminders individually or import timetable entries in bulk. Each record includes a title, type, start time, repeat setting and status.
5. The ESP32 saves changes to LittleFS. Its scheduler continuously checks pending reminders and determines whether an alert is due.
6. At the scheduled time, the OLED displays the reminder and the passive buzzer sounds. The user can press STOP or SNOOZE; MENU is used to navigate display pages when no alert is active.

5.3 Reminder Data and Persistence

Each reminder record contains an ID, title, type, start time in epoch seconds, repeat mode and status. Repeat mode supports once, daily and weekly schedules; status distinguishes pending, done and missed reminders. The firmware writes the JSON data to a temporary file before replacing the main file. This save procedure helps recover from an interruption during a write. On startup, the ESP32 loads saved records from LittleFS.

5.4 Local Wi-Fi Clarification

Wi-Fi is used only as a short-range communication link between the ESP32 hotspot and the user's phone or laptop. The phone's browser exchanges data with the ESP32's local web server. The Wi-Fi network does not provide internet access, and the ESP32 does not send reminder information to an external server. The phone is used to provide the current time when the dashboard is opened; this is a local time-setting step, not an internet time synchronisation service.

6. Hardware Components and Pin Configuration

The hardware combines an ESP32 DevKit, an I2C OLED display, a passive buzzer and three tactile push buttons. The wiring diagram below is reproduced from the supplied Smart Daily Reminder Assistant build guide (page 6).

6.1 Pin Mapping

Component	Component pin	ESP32 connection	Purpose
OLED (I2C)	VCC / GND	3V3 / GND	Power supply
OLED (I2C)	SDA / SCL	GPIO21 / GPIO22	I2C display communication
Passive buzzer	+ / –	GPIO14 / GND	Audible notification
STOP button	Two terminals	GPIO27 / GND	Acknowledge and stop alert
SNOOZE button	Two terminals	GPIO26 / GND	Postpone alert sequence
MENU button	Two terminals	GPIO25 / GND	Change OLED page
Optional DS3231	SDA / SCL	GPIO21 / GPIO22	Real-time clock upgrade

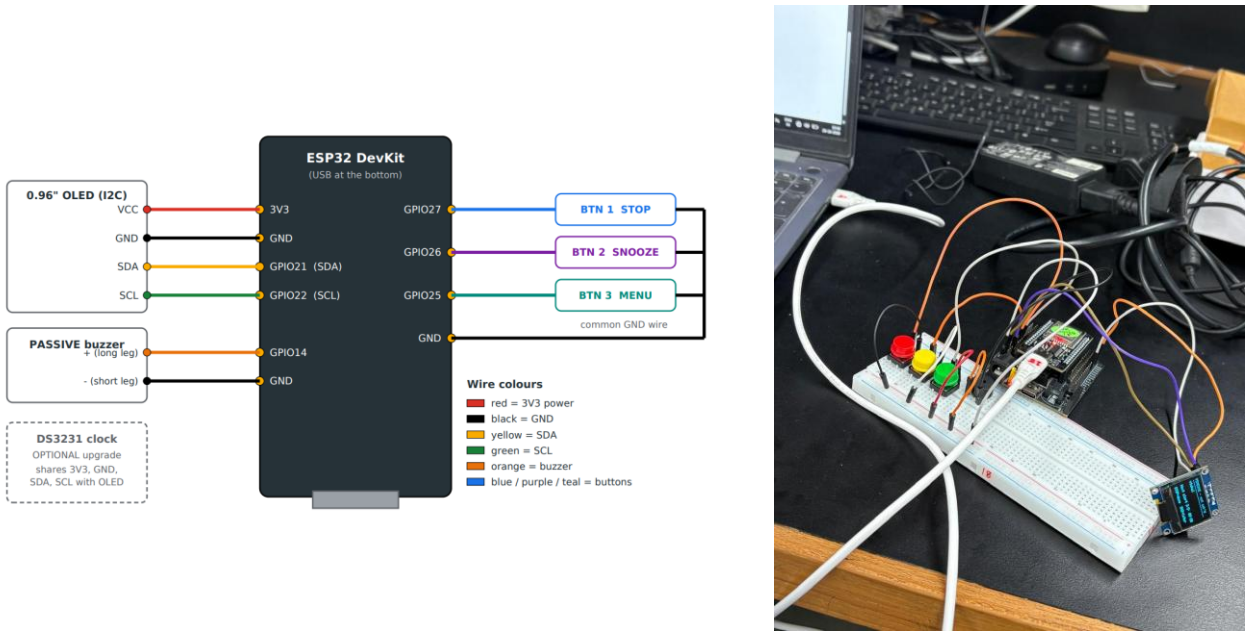


Figure 1. ESP32 pin/wiring diagram (left) and assembled project prototype (right).

6.2 Component Roles

ESP32 DevKit: runs the firmware, local Wi-Fi access point, web server, scheduler and peripheral control.

0.96-inch OLED (I2C): displays the next reminder, agenda, system status and alert messages via GPIO21 (SDA) and GPIO22 (SCL); I2C address 0x3C.

Passive buzzer: connected to GPIO14 and GND; ESP32 LEDC generates the three-beep alert pattern.

Push buttons: STOP, SNOOZE and MENU connect GPIO27, GPIO26 and GPIO25 to GND. Internal pull-ups make a press read LOW; 40 ms debounce filters contact chatter.

DS3231: shares the OLED I2C lines and retains time through power loss; not required for the basic build.

7. Software Design and Alert Algorithm

The firmware is developed in Arduino IDE for ESP32. The principal libraries identified in the guide are ArduinoJson 7.x for JSON handling, Adafruit SSD1306 for the OLED driver and Adafruit GFX for display drawing. WiFi, WebServer and LittleFS are supplied with the ESP32 Arduino core.

7.1 Alert Behaviour

The default alert configuration is three alarms per reminder, a five-minute interval between alarm starts, a 30-second ringing duration for each alarm and a ten-minute SNOOZE delay. If the user ignores all three alarms, the reminder is marked MISSED after 10 minutes and 30 seconds from the original alert start.

- **STOP:** immediately marks the current reminder DONE, silences the buzzer and prevents further alarms for that occurrence—even if pressed during a silent interval between alarms.
- **SNOOZE:** silences the current alert and restarts the full alert sequence ten minutes later.
- **MENU:** cycles through Next Reminder, Upcoming List and System Information. It is ignored while an alert is active.

7.2 Scheduling Logic

The scheduler uses elapsed time rather than relying on a separate alarm counter. Let elapsed = current time − alertBase. The alarm slot is calculated as elapsed / 300 seconds, and the phase within that slot is elapsed % 300. The buzzer is active only when the slot is one of the first three slots and the phase is below 30 seconds. The reminder becomes missed when elapsed reaches 630 seconds, unless STOP or SNOOZE has changed its state.

The main loop repeatedly handles the web server, reads buttons, updates the scheduler, updates the buzzer and refreshes the OLED. The buzzer pattern is generated without a long blocking delay, allowing the interface and controls to remain responsive.

8. Testing and Observed Behaviour

The supplied guide defines a 14-case test checklist. It covers startup before time is set, joining the hotspot, adding reminders, alarm activation, STOP during an alarm, ignoring all alerts, STOP during a silent interval, SNOOZE, OLED page navigation, weekly repetition, power cycling, simultaneous reminders, bulk import and clearing finished records.

Expected behaviour in these tests includes: no alarm before the clock is set; dashboard access through the local address; reminders appearing on the OLED; three 30-second alarms at five-minute intervals when ignored; permanent acknowledgement after STOP; a restarted sequence after SNOOZE; and reminder records remaining in flash after a power cycle. The guide also reports software simulation of key alert cases. These are the behaviours specified and tested in the supplied guide; hardware test results should be reported only for tests performed on the assembled device.

9. Limitations and Future Scope

The ESP32 has limited storage, and the guide sets a default capacity of 100 reminders. Without the optional DS3231, the ESP32's clock must be set again after power is removed by opening the local dashboard. Reminder editing is available only within the range of the device's hotspot, and there is no cloud backup. The guide recommends keeping a separate copy of timetable entries.

Possible extensions include adding the DS3231 RTC, battery or power-bank operation, a mute mode, reminder backup and restore, voice output, or Bluetooth Low Energy support. These are future enhancements and are not claimed as implemented features of the current version.

10. Conclusion

The Smart Daily Reminder Assistant applies edge computing to everyday scheduling. Local Wi-Fi, LittleFS storage, an on-device scheduler, OLED feedback, buzzer alerts and physical controls make the ESP32 a self-contained reminder platform without cloud or internet dependence.